

TREDENCE ASSIGNMENT

Self - Pruning Neural Network

Submitted by EDWIN RAPHAEL

Results Summary

Lambda (λ)	Test Accuracy (%)	Sparsity Level (%)
1e-8	54.56	8.57
1e-4	50.18	84.97
0.1	36.85	88.01

```
###
#imports
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
###
#defining the prunable layer
class PrunableLinear(nn.Module):
    def __init__(self, in_features, out_features):
        super(PrunableLinear, self).__init__()

        # Standard parameters
        self.weight = nn.Parameter(torch.randn(out_features, in_features) * 0.01)
        self.bias = nn.Parameter(torch.zeros(out_features))

        # Learnable gate scores
        self.gate_scores = nn.Parameter(torch.randn(out_features, in_features) - 2.5)

    def forward(self, x):
        # Convert scores → gates (0 to 1)
        gates = torch.sigmoid(self.gate_scores)
```

```

# Apply gating Element-wise masking of weights
pruned_weights = self.weight * gates

# Linear transformation
return F.linear(x, pruned_weights, self.bias)

#model
class PrunableNet(nn.Module):
    def __init__(self):
        super(PrunableNet, self).__init__()

        self.fc1 = PrunableLinear(32*32*3, 1024)
        self.fc2 = PrunableLinear(1024, 512)
        self.fc3 = PrunableLinear(512, 256)
        self.output_layer = PrunableLinear(256, 10)

    def forward(self, x):
        x = x.view(x.size(0), -1)

        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = F.relu(self.fc3(x))
        x = self.output_layer(x)

        return x

#dataset import
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                       download=True, transform=transform)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=128, shuffle=False)

#sparsity loss
def compute_sparsity_loss(model):
    loss = 0.0

```

```

for module in model.modules():
    if isinstance(module, PrunableLinear):
        gates = torch.sigmoid(module.gate_scores)
        #sum of gate values
        loss += gates.sum()

return loss

#model training
import tqdm

def train_model(model, trainloader, lambda_sparse, epochs=10):
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        # tqdm over batches
        loop = tqdm.tqdm(trainloader, desc=f"Epoch {epoch+1}/{epochs}", leave=False)

        for images, labels in loop:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()

            outputs = model(images)

            classification_loss = F.cross_entropy(outputs, labels)
            sparsity_loss = compute_sparsity_loss(model)

            loss = classification_loss + lambda_sparse * sparsity_loss
            #Gradients flow through both weight and gate_scores because sigmoid is differentiable,
            #allowing the model to learn which connections to suppress.
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        # update progress bar
        loop.set_postfix(loss=loss.item())

    print(f"Epoch {epoch+1}, Total Loss: {total_loss:.4f}")

```

#evaluation

```
def evaluate_accuracy(model, testloader):
    model.eval()
    correct, total = 0, 0

    with torch.no_grad():
        for images, labels in testloader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (preds == labels).sum().item()

    return 100 * correct / total
```

#sparsity metric

```
def compute_sparsity(model, threshold=0.02):
    total, pruned = 0, 0

    for module in model.modules():
        if isinstance(module, PrunableLinear):
            gates = torch.sigmoid(module.gate_scores)

            total += gates.numel() #num of elements
            pruned += (gates < threshold).sum().item()

    return 100 * pruned / total
```

#graph plot

```
def plot_gate_distribution(model):
    all_gates = []

    for module in model.modules():
        if isinstance(module, PrunableLinear):
            gates = torch.sigmoid(module.gate_scores).detach().cpu().numpy()
            all_gates.extend(gates.flatten())

    plt.figure(figsize=(8, 5))

    plt.hist(all_gates, bins=50, range=(0, 1))

    plt.title("Distribution of Learned Gate Activations (Pruning Behavior)")
    plt.xlabel("Gate Activation Value (0 = Pruned, 1 = Fully Active)")
```

```

plt.ylabel("Number of Weights (Frequency)")

plt.grid(alpha=0.3)

plt.show()

#train eval loop

lambdas = [1e-8, 1e-4, 1e-1]
results = []
trained_models = [] # ONLY ADDITION

for lam in lambdas:
    print(f"\n ♦ Training with lambda = {lam}")

    model = PrunableNet().to(device)

    train_model(model, trainloader, lam, epochs=10)

    acc = evaluate_accuracy(model, testloader)
    sparsity = compute_sparsity(model)

    results.append((lam, acc, sparsity))
    trained_models.append((lam, model)) # ONLY ADDITION

    print(f"Lambda: {lam} | Accuracy: {acc:.2f}% | Sparsity: {sparsity:.2f}%")

#Results
print("\nFinal Results:")
print("Lambda\tAccuracy\tSparsity")

for lam, acc, sparsity in results:
    print(f"{lam}\t{acc:.2f}%\t{sparsity:.2f}%")

# Plot gate distributions for each trained model
for lam, model in trained_models:
    print(f"\n 🚪 Gate Distribution for  $\lambda = {lam}$ ")
    plot_gate_distribution(model)

```